Introduction to Web Technologies — Midterm Project

The same project, finished: complete logic, every element in place, and markup ready for JavaScript

What the midterm is

There is no new topic this time. You take the project you have been building since Assignment 1 — same team, same organization, same repository — and you finish it as a whole. **The midterm is graded on completeness and coherence, not on new features.** Most of the work is finishing what is already half-done, making the separate pages behave like one site, and fixing what was left broken in earlier weeks. **Build it as if it were a real project you were handing over to the organization** — a site they could put online tomorrow and not be embarrassed by. Not a university exercise, not a demo, not something half-built that you explain away in class: a finished website for a real place.

No JavaScript yet — you are preparing the ground

You are not writing any JavaScript this time, and nothing has to actually work yet: a button does not have to react when it is clicked and a form does not have to produce a real answer. **What is required is that every element and every style your site will need already exists and is finished** , so that in the following assignments JavaScript only has to bring them to life. You are building the stage and the props; the acting comes later.

Go through everything again and add what is missing

Do not only polish what is already there. Open every page and ask what the visitor needs here that you never built. **If the logic calls for a form, build the form now** — a booking, an order, a feedback box, a search field. If a price list needs a place for a total, if a service needs a button that starts the booking, if a gallery needs captions, if a submitted form needs somewhere for the confirmation to appear, add those elements now. The same goes for styles: anything that will later be shown, hidden, highlighted, selected or marked as an error must already be styled, even though nothing triggers it yet. Think every small detail through — after this midterm you cannot add it by hand.

The main requirement: the logic must finish

I see the same problem in almost every project: the logic starts but never ends. A menu lists dishes but there is nowhere to order them. A booking form exists but the visitor is never told what happens next. A page links to a section that does not exist. **Walk through your site as a real visitor and make sure every path you start, you also finish.** Concretely: every link leads to a real page or a real section on the page — no dead links, no href="#", no "coming soon"; every page is reachable from every other page through the navigation; every page is complete, with no empty blocks left for later; every form tells the visitor what will happen when they submit it and has a visible place where the result or confirmation will appear; every list, table and price block has real, finished content. Think every detail through before you code — the small missing pieces are what cost the most points.

Three user journeys

In your README, describe three journeys **one visitor** would take through your site — for example "find the opening hours and the address", "compare two services and book one", "read about the team and send a question". Write the start, the steps and the end of each. **All three must work end to end when I click through them.** This is the fastest way to find out yourself whether your logic finishes.

One visitor, one site — keep the scope realistic

Everything your site offers must be something **a single visitor can complete alone, from their own screen** , by interacting with your pages. If your project is about ordering, one person places the order: you do not need a second person to confirm it, a manager to approve it or a courier to mark it delivered. Do not design staff accounts, admin panels, employee dashboards or approval chains — you have no server and no database, so those paths can never be finished and they are exactly what leaves projects half-done. The interaction you are building runs between the visitor and your pages: choosing something, moving to another page, filling a form in, submitting it and being told the result.

Prepare the markup for JavaScript — this is a freeze

From the next assignment onward you will not add HTML elements or write new CSS by hand. Everything new — new content, new states, new behaviour — will be created and changed through JavaScript. **So whatever your site will need later has to exist now.** Before you submit, go through your pages and prepare the hooks: give an id to every element JavaScript will need to find — every form, every input, every button, every block that will change; keep your ids and class names in English, lowercase and consistent across the project; add an empty container with an id wherever a list of items will later be generated or a message will be shown, for example a results area, an error area, a confirmation area; and write the CSS classes that describe states in advance — hidden, active, selected, error, success — even though nothing uses them yet, because later JavaScript will only switch classes on and off.

One site, not three

Teams often submit three pages that look like three different websites. Agree on and apply one set of rules across every page: the same navigation in the same place with the same items, the same header and footer, one title pattern, one set of button styles and colours, one way of naming files, classes and ids. Nothing on a page should reveal which team member wrote it.

The technical floor

Every page passes the W3C validator with zero errors. The browser console shows no errors on any page. No broken images and no broken links anywhere. Nothing overflows sideways and no horizontal scrollbar appears at phone width. Bootstrap still

does the layout and your own CSS stays a small correction layer. All content remains real — real prices, hours, address, phone, your own photographs; no placeholder text, no stock or AI-generated images.

Quality pass before you submit

At least two days before the deadline, each student opens the pages written by someone else in the team — or, if you work alone, opens your site on a different device — clicks every link, submits every form and writes down what is broken. Fix the findings and keep the short list of what you found and fixed. **Submitting without this pass is obvious: it is exactly the projects with dead links and empty blocks.**

Freeze your structure

When everything is finished, make a final commit and tag it in Git with the name midterm. That tag is the frozen state of your HTML and CSS. In the assignments that follow I will compare your repository against it, and new elements or new hand-written styles appearing after the tag will not count — they must be produced by JavaScript instead. Make the structure good now; you will live inside it for the rest of the course.

Submission

The same Git repository with the midterm tag, containing the finished pages, your stylesheet, the three user journeys and the page list in the README, the quality-pass list, the AI log, and screenshots of each page at phone and desktop width. Everyone commits from their own account across at least four different days.

Defense

I will click through your site and your three journeys in front of you and stop wherever the logic breaks. I will ask each student to explain a page they did not write, to explain which elements they prepared for JavaScript and why, and to justify a class or a Bootstrap choice. **I may also give you a small practical task on any topic we have covered so far** — HTML structure, semantic tags, tables, forms, CSS selectors and specificity, the box model, Bootstrap grid or utilities — and you write it in front of me to show that you can. On top of that there will be a few oral questions from the lectures and the week's topics. **During the defense you may look at the official documentation and at w3schools.com** — nobody is expected to remember every attribute by heart. What you may not do is ask another person or an AI, and looking something up does not replace understanding it: you still have to explain what you looked up and why it fits. Your grade is the project score multiplied by how well you explain your own work; if you cannot explain it, the score is zero. **Work that is submitted but not defended is not evaluated, even if it was uploaded before the deadline.**

Grading (100 points)

Logical completeness and the three journeys 25 · nothing missing: the elements the logic needs are all built 18 · readiness for JavaScript: ids, empty containers, state classes, consistent naming 15 · one consistent site across all pages 12 · technical floor: validator, console, links, responsive 12 · real content and no placeholders 9 · quality pass, freeze tag and commit history 9. Penalties: 10 for a flow that one visitor cannot finish alone, 10 per empty or "coming soon" page, 10 for placeholder text or a stock image, 10 for horizontal overflow at phone width, 5 per dead or href="#" link, 5 per console error, 5 for navigation that differs between pages, 5 for a missing freeze tag.

AI policy

Unchanged: you may ask AI to explain concepts, and every question goes in the AI log. You may not have it write your pages. What is graded here is judgement and attention to detail — whether you noticed the missing confirmation, the dead link and the block you never finished — and that is checked by clicking through your site with you standing next to it.